

```

nn(m_swap, X,[0,1]) :: swap(X,Y,0) ; swap(X,Y,1).

hole(X,Y,X,Y):-
    swap(X,Y,0).

hole(X,Y,Y,X):-
    swap(X,Y,1).

bubble([X],[ ],X).
bubble([H1,H2|T],[X1|T1],X):-
    hole(H1,H2,X1,X2),
    bubble([X2|T],T1,X).

bubblesort([ ],L,L).

bubblesort(L,L3,Sorted):-
    bubble(L,L2,X),
    bubblesort(L2,[X|L3],Sorted).

sort(L,L2):- bubblesort(L,[ ],L2).

```

Listing 4: Forth sorting sketch (**T4**)

argument. The bubblesort/3 predicate uses the bubble/3 predicate, and recursively calls itself on the remaining list, adding the last element on each step to the front of the sorted list.

```

permute(0,A,B,C,A,B,C).
permute(1,A,B,C,A,C,B).
permute(2,A,B,C,B,A,C).
permute(3,A,B,C,B,C,A).
permute(4,A,B,C,C,A,B).
permute(5,A,B,C,C,B,A).

swap(0,X,Y,X,Y).
swap(1,X,Y,Y,X).

operator(0,X,Y,Z):- Z is X+Y.
operator(1,X,Y,Z):- Z is X-Y.
operator(2,X,Y,Z):- Z is X*Y.
operator(3,X,Y,Z):- Y > 0, 0 == X mod Y,Z is X//Y.

nn(m_net1, Repr, [0,...,6]::net1(Repr,0);...;net1(Repr,6).
nn(m_net2, Repr, [0,...,3]::net2(Repr,0);...;net2(Repr,3).
nn(m_net3, Repr, [0,1]::net3(Repr,0);net3(Repr,1).
nn(m_net4, Repr, [0,...,3]::net4(Repr,0);...;net4(Repr,3).

wap(Text,X1,X2,X3,Out):-
    net1(Text,Perm),
    permute(Perm,X1,X2,X3,N1,N2,N3),
    net2(Text,Op1),
    operator(Op1,N1,N2,Res1),
    net3(Text,Swap),
    swap(Swap,Res1,N3,X,Y),
    net4(Text,Op2),
    operator(Op2,X,Y,Out).

```

Listing 5: Forth WAP sketch (**T5**)

In Listing 5, there are four neural predicates: net1/2 to net4/2. Their first argument is the input question, and the second argument are indicator variables for the choice of respectively: one of six permutations, one of 4 operations, swapping and one of 4 operations. These are implemented in the

permute/7, swap/5 and operator/4 predicates. The wap/5 predicate then sequences these steps to calculate the result.

```

nn(m_colour,R,G,B,[red,green,blue]::colour(R,G,B,red);
                                   colour(R,G,B,green);colour(R,G,B,blue).

nn(m_coin,Coin,[heads,tails])::coin(Coin,heads);coin(Coin,tails).

t(0.5)::col(1,red);t(0.5)::col(1,blue).
t(0.333)::col(2,red);t(0.333)::col(2,green);t(0.333)::col(2,blue).
t(0.5)::is_heads.

outcome(heads,red,-,win).
outcome(heads,-,red,win).
outcome(_,C,C,win).
outcome(Coin,Colour1,Colour2,loss):-\+outcome(Coin,Colour1,Colour2,win).

game(Coin,Urn1,Urn2,Result):-
    coin(Coin,Side),
    urn(1,Urn1,C1),
    urn(2,Urn2,C2),
    outcome(Side,C1,C2,Result).

urn(ID,Colour,C):-
    col(ID,C),
    colour(Colour,C).

coin(Coin,heads):-
    coin(Coin,heads),
    is_heads.

coin(Coin,tails):-
    coin(Coin,tails),
    \+is_heads.

```

Listing 6: The coin-ball problem (T6)

In Listing 6, there are two neural predicates: colour/4 and coin/2. There are also 6 learnable parameters: 2 for the first urn, 3 for the second and one for the coin. The outcome/4 defines the winning conditions based on the coin and the two urns. The urn/3 and coin/2 predicates tie the parameters to the detections of the neural predicates. The game predicate is the high-level predicate that plays the game.